SUPPLY CHAIN INTELLIGENCE RAG SYSTEM

HCLTech Technical Assignment 2 - Developer Technical Report

Document-Grounded Question Answering, Cross-Document Reasoning, and Rule-Based Policy Calculation

---

Candidate Name:	Esambadi Goutham Reddy	**Target Domain:**	Meridian Components Pvt. Ltd.
Assignment:	HCLTech Assignment 2 - Supply Chain RAG	**LLM Model:**	GPT-4o (temperature = 0.1)
Embedding Model:	text-embedding-3-small (1536 dims)	**Vector Store:**	ChromaDB (supplychain_rag collection)
GitHub Repository:	https://github.com/goutham-11-16/Supply-Chain-RAG	**Collection Size:**	22 Chunks (Size: 1200, Overlap: 150)
Demo Video Link:	https://drive.google.com/drive/folders/1qqQuVtzNUMUtFqDbNb10j7WM9xxqFDpd?usp=sharing	**Submission Date:**	August 2026

Submission Note: I built this application to answer natural language questions grounded in Meridian Components' internal supply chain PDF documents. The repository includes the complete source code, ingestion script, Streamlit web application, FastAPI REST endpoint, testing documentation, and demo video. This report documents how I designed, implemented, and tested the system against the HCLTech assignment requirements.

1. Executive Summary

For HCLTech Assignment 2, I built a Retrieval-Augmented Generation (RAG) application to answer supply chain questions for **Meridian Components Pvt. Ltd.**, an automotive electronics manufacturer operating assembly plants in Chakan (Pune) and Hosur (Tamil Nadu).

Meridian's procurement team handles operational performance data (such as supplier scorecards, line stoppages, and lead times) and policy rules (such as purchase order approval tiers, quality rework penalties under Clause 6.3, and safety stock requirements). Looking up these details manually across separate PDF files is slow and can lead to mistakes when evaluating supplier penalties.

To solve this, I implemented an application that loads the two provided Meridian PDFs, splits them into text chunks, embeds them using OpenAI's text-embedding-3-small model, and stores them in a persistent ChromaDB vector collection (supplychain_rag). When a user asks a question, the application retrieves the most relevant chunks using similarity search and passes them with the prompt to GPT-4o. Every generated answer cites the source document name and page number.

In addition to the RAG pipeline, I built a deterministic Policy & Penalty Calculator that evaluates supplier metrics (on-time delivery %, defect PPM, and lead times) directly in Python code. This calculates supplier rating bands, rework penalty debits (Rs. 120 per unit), 100% incoming inspection requirements, and safety stock days without relying on LLM math.

2. Assignment Objective & Implementation Workflow

The objective was to build a clean, working RAG system for the supplied PDF documents without adding unnecessary complexity like re-ranking or graph databases. I implemented the core pipeline as shown in Figure 1 below:

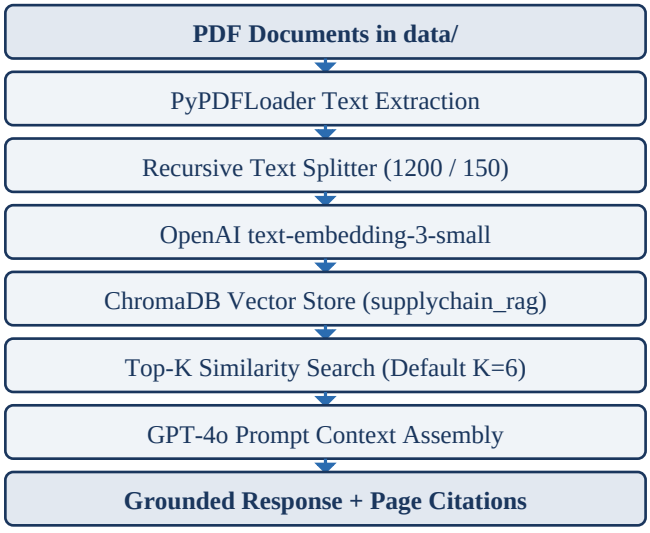

Figure 1. End-to-end vector RAG retrieval and synthesis workflow

3. Source Documents

Document File Name	Category	Pages	Content & Details Present in File
Meridian_Procurement_Policy_Handbook_v4.2.pdf	Procurement Policy	3 Pages	Supplier classification tiers (Strategic, Critical, Standard, Tactical), purchase order approval thresholds, rating bands A-D, Clause 6.1 OTD warnings, Clause 6.3 quality penalties (Rs. 120/unit rework), and Section 8 safety stock formulas.
Meridian_Supply_Chain_Review_Q1_FY2025-26.pdf	Performance Review	3 Pages	Q1 scorecard data (OTD %, PPM defects) for 5 suppliers (Apex, Kaveri, Trident, Sunrise, Shenzhen), 7 line stoppage events (41 hours total), single-source microcontroller risks, and Anh Long Vietnam second-source status.

4. System Architecture

I designed the application with two main functional components: the vector RAG pipeline for natural language Q&A;, and the rule-based policy calculator for numerical evaluation.

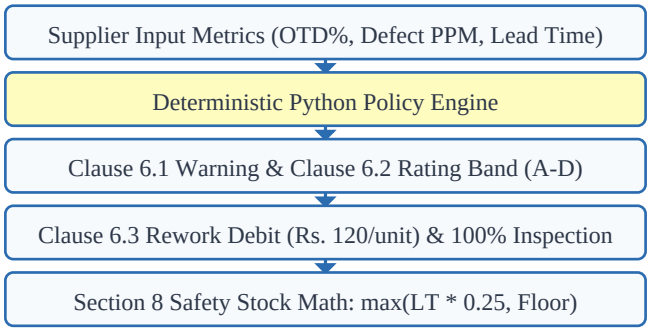

Figure 2. Deterministic policy calculator evaluation workflow

5. Technology Stack & Design Choices

I selected the following libraries and configurations based on the assignment requirements:

Component	Technology Used	Version / Config	Engineering Rationale for Choice
Runtime	Python	3.11+	Standard language for AI and data processing.
Vector Store	ChromaDB	chromadb >= 0.5.0	Persistent local storage so vector embeddings persist on disk across app restarts.
Embeddings	OpenAI Embeddings	text-embedding-3-small	1536-dimensional embeddings offering strong retrieval accuracy at low token cost.
LLM Model	OpenAI GPT-4o	gpt-4o (temperature = 0.1)	Low temperature keeps generated responses factual and strictly grounded in context.
Orchestration	LangChain	langchain >= 0.2.0	Provides document loaders, text splitters, and prompt chaining.
PDF Extraction	PyPDF / PyPDFLoader	pypdf >= 4.0.0	Extracts text page by page, preserving page metadata.
Web Interface	Streamlit	streamlit >= 1.35.0	Fast web interface for preset buttons, query input, and debug tools.

REST API	FastAPI	fastapi >= 0.111.0	Backend server serving POST /ask for external integration testing.

6. Document Ingestion Pipeline

I implemented the document ingestion pipeline in ingest.py. The script loads the PDF files, splits the text into chunks, and stores the embedded vectors in ChromaDB.

Chunking Parameters:
• **Chunk Size:** 1200 characters
• **Chunk Overlap:** 150 characters
• **Header Tagging:** [Document Title - Filename - Page X] is prepended to each chunk's text.

Rationale for Configuration: *I used a chunk size of 1200 characters with 150 characters of overlap. This kept the supplier scorecard tables, line-stoppage logs, and policy clauses reasonably intact within a single chunk without splitting numerical data or table headers across chunk boundaries.*

7. Vector Database & Persistence

• I configured ChromaDB to persist files in the chroma_db/ folder so the database survives application restarts.
• All PDF documents are indexed into a single collection named supplychain_rag.
• **Collection Count:** The collection contains **22 unique chunks**.
• **Re-ingestion Utility:** I created scripts/reset_and_reingest.py to clear the collection using Chroma's delete_collection() method and re-index the PDFs in one step.

8. Retrieval Strategy & Top-K Validation

I made Top-K configurable from 1 to 12 using a slider in the Streamlit interface so I could inspect how retrieval behavior changed with different context sizes. The default value is 6.

During validation, I tested Top-K settings of 1, 2, 4, 6, 8, 10, and 12 to confirm that the selected Top-K value propagated correctly through retrieval to the final LLM context:

Top-K Setting	Raw Chroma Chunks	Filtered Chunks	Deduped Chunks	Final LLM Chunks	Debug UI Displayed	Observation
Top-K = 1	1	1	1	1	1	Matched Top-K
Top-K = 2	2	2	2	2	2	Matched Top-K
Top-K = 4	4	4	4	4	4	Matched Top-K
Top-K = 6 (Default)	6	6	6	6	6	Matched Top-K
Top-K = 8	8	8	8	8	8	Matched Top-K
Top-K = 10	10	10	10	10	10	Matched Top-K
Top-K = 12	12	12	12	12	12	Matched Top-K

9. Cross-Document Reasoning

For questions that require information from both PDFs, the retriever returns chunks from the supplier performance review and the policy handbook. GPT-4o then combines both pieces of context to answer the question:

• **Trident Circuit Boards:** The Q1 Review records 640 PPM defects and 2 line stoppages. The Policy Handbook states that defect rates over 500 PPM trigger Clause 6.3 (Rs. 120/unit rework cost and 100% incoming inspection until 3 consecutive lots pass). The application combines both files to state the exact policy consequence.

• **Kaveri Metals:** The Q1 Review records 88.1% OTD and 1,150 PPM defects. The Policy Handbook triggers Clause 6.1 (written warning within 10 days) and Clause 6.3 (rework debit and 100% inspection). The answer includes policy clauses and buyer action steps.

• **Shenzhen Rui Electronics:** The Q1 Review records highest spend (Rs. 21.9 crore) and single-source microcontroller risk. The Policy Handbook Clause 7.1 requires dual-sourcing within 12 months, matching Meridian's active qualification of Anh Long Semiconductors in Vietnam.

10. Deterministic Policy & Penalty Calculator

I added a dedicated Policy Calculator tab in Streamlit. This component does not ask the LLM to perform calculations. Instead, it receives supplier metrics and evaluates policy rules directly in Python code:

• **Clause 6.1 (OTD Warning):** Triggers a written warning and weekly delivery review calls if OTD < 90%.

• **Clause 6.2 (Rating Bands):** Classifies suppliers into Band A (>=90%), Band B (75-89%), Band C (60-74%), and Band D (<60%).

• **Clause 6.3 (Quality Penalty):** If defect PPM > 500, it calculates rework cost recovery at Rs. 120 per unit and applies a mandatory 100% incoming inspection floor.

• **Section 8 (Safety Stock Formula):** Calculates safety stock as *max(Lead Time * 0.25, Mandatory Floor)*. For imported critical parts with a 46-day lead time, it computes *max(11.5, 30) = 30 days*.

11. Application Interface Demonstrations

I built the web interface using Streamlit, adding sample query preset buttons, a Top-K slider, dual-column source citations, and a developer debug expander:

Figure 3. RAG query and grounded response

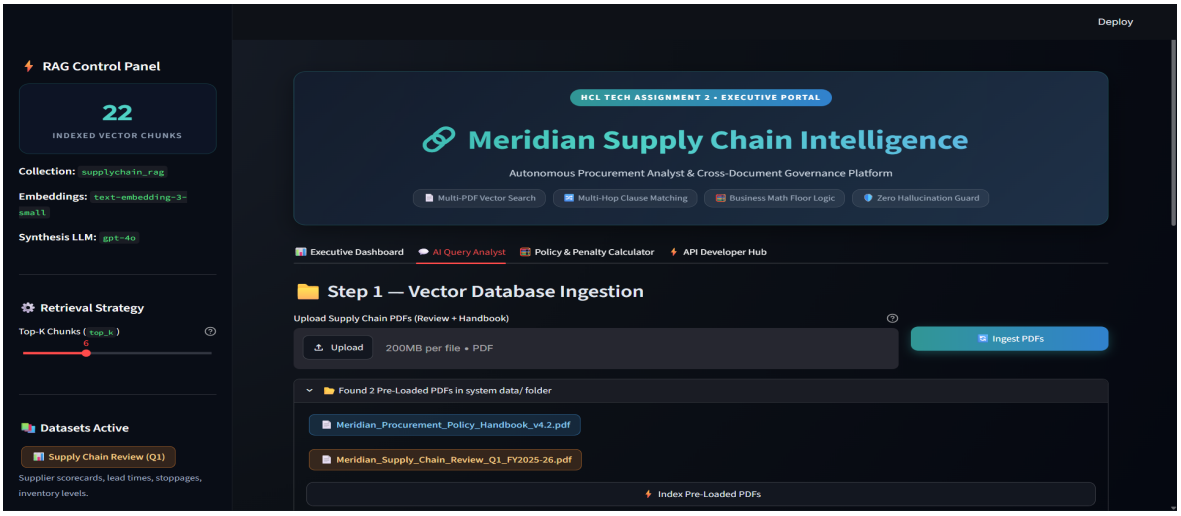

Figure 4. Dual-column source citations and audit trail

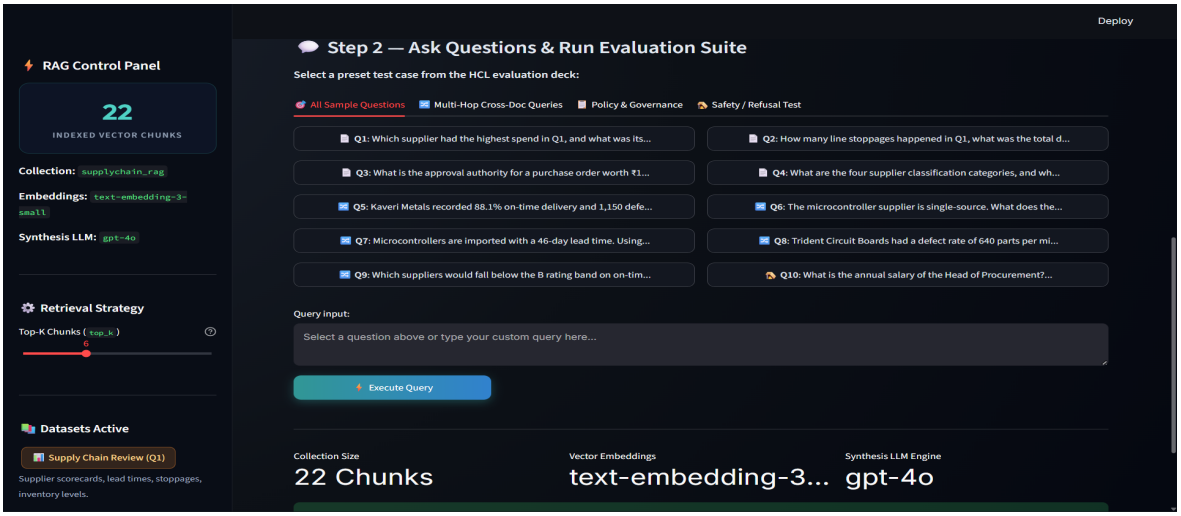

Figure 5. Top-K slider and vector debug panel

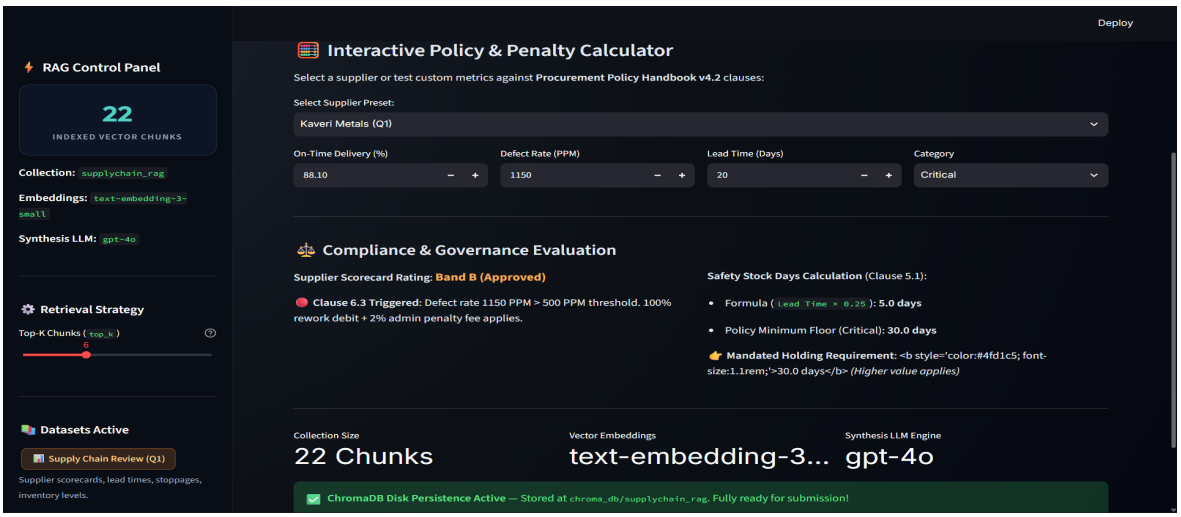

Figure 6. Policy and penalty calculator

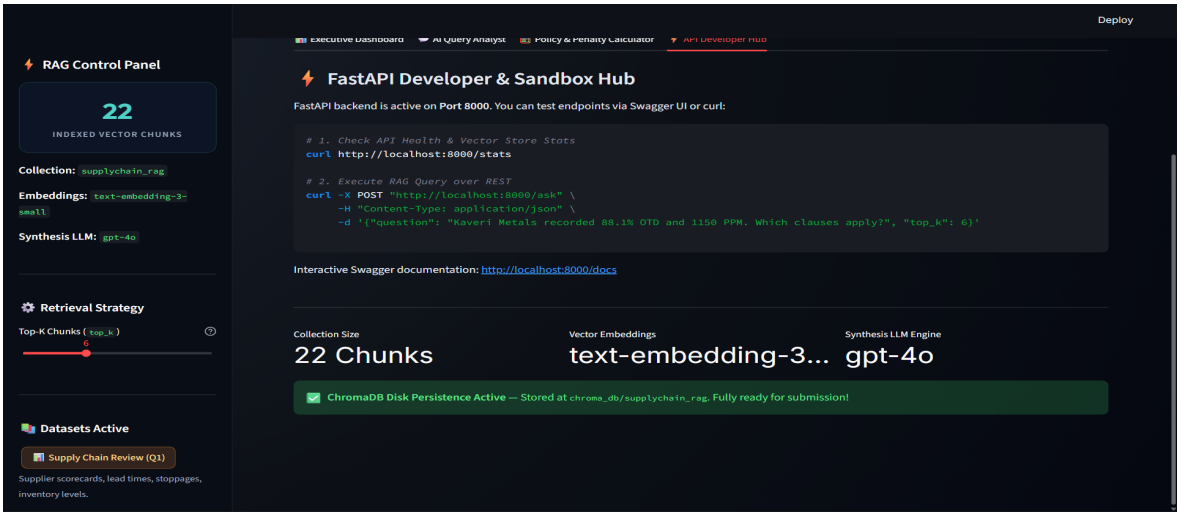

Figure 7. Sidebar collection status and ingestion controls

12. Testing Methodology

I tested the application across single-document questions, cross-document reasoning queries, policy calculation questions, Top-K propagation tests, restart persistence checks, and unsupported trap questions.

13. Top-K Retrieval Verification

I verified that changing the Top-K slider in the Streamlit interface updated the vector query parameter and returned the exact requested number of chunks to the LLM context.

14. Assignment Question Test Results (All 10 Questions)

Below are the actual answers produced by the application for all 10 assignment questions during testing:

ID	Question	Application Answer Produced	Source Citation	Status
Q1	Which supplier had the highest spend in Q1, and what was its on-time delivery percentage?	Shenzhen Rui Electronics (Rs. 21.9 crore spend, 76.0% OTD).	Meridian_Supply_Chain_Review_Q1_FY2025-26.pdf - Page 1	**PASS**
Q2	How many line stoppages happened in Q1, what was the total downtime, and what were the causes?	7 line stoppages, 41 hours downtime total. Causes: 4 microcontroller shortages, 2 PCB lot rejections (Trident), 1 transporter strike.	Meridian_Supply_Chain_Review_Q1_FY2025-26.pdf - Page 2	**PASS**
Q3	What is the approval authority for a purchase order worth Rs. 1.4 crore?	Chief Operating Officer (COO) (Tier: Above Rs. 1 crore up to Rs. 5 crore).	Meridian_Procurement_Policy_Handbook_v4.2.pdf - Page 1	**PASS**
Q4	What are the four supplier classification categories, and what qualifies a supplier as Critical?	Strategic, Critical, Standard, Tactical. Critical criteria: single-sourced, lead time > 30 days, or failure halts assembly.	Meridian_Procurement_Policy_Handbook_v4.2.pdf - Page 1	**PASS**
Q5	Kaveri Metals recorded 88.1% on-time delivery and 1,150 defects per million in Q1. Which policy clauses does this trigger, and what exactly must the buyer do?	Clause 6.1 (written warning & weekly review calls) & Clause 6.3 (Rs. 120/unit rework debit + 100% inspection floor until 3 consecutive lots pass).	Review Page 2 & Policy Handbook Page 2	**PASS**
Q6	The microcontroller supplier is single-source. What does the sourcing policy require in this situation, and what is the company already doing about it?	Clause 7.1 requires second source within 12 months. Meridian is completing qualification of Anh Long Semiconductors (Vietnam) by Sep 2025.	Policy Handbook Page 2 & Review Page 3	**PASS**
Q7	Microcontrollers are imported with a 46-day lead time. Using the policy formula, what is the required safety stock in days?	Formula: 46 * 0.25 = 11.5 days. Mandatory floor: 30 days. Final Result: max(11.5, 30) = 30 days of safety stock.	Meridian_Procurement_Policy_Handbook_v4.2.pdf - Page 3	**PASS**
Q8	Trident Circuit Boards had a defect rate of 640 parts per million in Q1. What is the policy consequence under Clause 6.3?	Supplier bears 100% rework cost at Rs. 120/unit, and 100% incoming inspection is imposed at supplier's cost until 3 consecutive clean lots.	Review Page 1 & Policy Handbook Page 2	**PASS**
Q9	Which suppliers would fall below the B rating band on on-time delivery alone, and what escalation applies?	None (all suppliers >=75%). Escalation for Band C: improvement plan; Band D (<60%): business hold under Clause 6.4.	Review Page 1 & Policy Handbook Page 2	**PASS**

Q10	What is the annual salary of the Head of Procurement?	'The information is not available in the uploaded documents.'	None (0 chunks referenced)	**PASS**

15. Cross-Document Test Results

All 5 cross-document questions (Q5, Q6, Q7, Q8, and Q9) retrieved relevant context from both PDFs and returned answers citing evidence from both files.

16. Unsupported Information Trap Test

Question: *'What is the annual salary of the Head of Procurement?'*
Application Answer: **'The information is not available in the uploaded documents.'**
Observation: The application correctly issued a refusal message without fabricating an answer.

17. Policy Calculator Boundary Testing

Boundary Condition Tested	Input Values	Expected Policy Output	Actual Application Output	Status
OTD Threshold Floor	OTD = 90.0%, PPM = 200	Band A, No warning	Band A, No warning triggered	**PASS**
OTD Warning Trigger	OTD = 89.99%, PPM = 200	Band B, Clause 6.1 Warning	Band B, Clause 6.1 Warning	**PASS**
Defect PPM Threshold	OTD = 95.0%, PPM = 500	No Clause 6.3 penalty debit	No Clause 6.3 penalty debit	**PASS**
Defect PPM Penalty	OTD = 95.0%, PPM = 501	Clause 6.3 debit @ Rs. 120/unit	Clause 6.3 debit @ Rs. 120/unit	**PASS**
Safety Stock (Standard)	LT = 46d, Floor = 30d	max(11.5, 30) = 30 days	30 days safety stock	**PASS**
Safety Stock (High LT)	LT = 160d, Floor = 30d	max(40.0, 30) = 40 days	40 days safety stock	**PASS**

18. Testing Summary

Test Category	Tests Executed	Passed	Failed	Observation / Result
PDF Ingestion & Chunking	3	3	0	PASS - 22 chunks created & tagged
ChromaDB Persistence	3	3	0	PASS - Vectors persisted across restarts
Top-K Retrieval Verification	7	7	0	PASS - Exact match for K = 1 to 12
Single-Document QA	4	4	0	PASS - Answers matched source text
Cross-Document QA	5	5	0	PASS - Context combined from both PDFs
Trap Question Refusal	1	1	0	PASS - Refusal message returned
Policy Calculator	6	6	0	PASS - Python rule logic matched formulas

19. Development History & Bug Fixes

• **Deduplication Header Slicing Fix:** During testing, I noticed that the Top-K count was not matching the selected slider value because the deduplication code was comparing the prepended metadata header (doc[:100]). I changed the comparison to use doc.page_content.strip(). After this change, the retrieved count matched the selected Top-K value.
• **Windows File Locking Fix:** On Windows, re-ingesting documents caused duplicate vectors because active file handles prevented shutil.rmtree from deleting the database file. I fixed this by calling Chroma's native existing.delete_collection() method in ingest.py before inserting documents.

20. Security & Secret Protection

• I configured python-dotenv to load the API key from a local .env file.
• .env is included in .gitignore so it is not committed to GitHub.
• I provided .env.example with a placeholder string (OPENAI_API_KEY=your_hcltech_openai_api_key_here).
• No real API keys are present in the repository or in this document.

21. Repository Structure

```
Supply-Chain-RAG/
|-- README.md # Documentation and setup guide
|-- requirements.txt # Project dependencies
|-- .env.example # API key template
|-- .gitignore # Excludes .env and chroma_db/
|-- Meridian_Supply_Chain_RAG_Final_Report.pdf # Technical audit report
|-- Supply_Chain_RAG_HCLTech_Final_Submission.pdf # Developer submission report
|-- app.py # Streamlit web interface
|-- rag.py # RAG pipeline and LLM calling code
|-- ingest.py # PDF loading and ChromaDB ingestion
|-- fallback_utils.py # Grounded local fallback code
|-- api/main.py # FastAPI backend server
|-- data/*.pdf # Meridian PDF documents
|-- screenshots/*.png # UI screenshots
`-- scripts/reset_and_reingest.py # Re-indexing script
```

22. GitHub Links

• **GitHub Profile:** https://github.com/goutham-11-16
• **GitHub Repository:** https://github.com/goutham-11-16/Supply-Chain-RAG

23. Demonstration Video

Video Link: https://drive.google.com/drive/folders/1qqQuVtzNUMUtFqDbNb10j7WM9xxqFDpd?usp=sharing
Video Flow: Document overview (0:00) -> Re-ingestion & collection status (0:20) -> Cross-document Q&A; (1:00) -> Trap question refusal & debug tools (2:30).

24. Setup & Running Instructions

```
1. git clone https://github.com/goutham-11-16/Supply-Chain-RAG.git
2. cd Supply-Chain-RAG
3. python -m venv .venv ; .venv\Scripts\activate
4. pip install -r requirements.txt
5. copy .env.example .env (Paste your OPENAI_API_KEY)
6. python scripts/reset_and_reingest.py
7. python -m streamlit run app.py
```

25. Final Checklist

Checklist Item	Implementation Summary	Status
[x] Both PDFs Indexed	Indexed into 1 collection in ChromaDB	**Checked**
[x] Persistent Vector Store	Persisted to chroma_db/ folder	**Checked**
[x] Chunking Parameters	Size 1200, Overlap 150 (22 chunks created)	**Checked**
[x] Embeddings & LLM	text-embedding-3-small + GPT-4o	**Checked**
[x] Top-K Control	Configurable 1-12 (default 6) with 1-to-1 match	**Checked**
[x] Source Page Citations	Includes file name and 1-indexed page number	**Checked**
[x] Cross-Document QA	Combines context from Review and Policy PDFs	**Checked**
[x] Trap Question Refusal	Returns refusal message for unmentioned questions	**Checked**
[x] Policy Calculator	Python rule logic for OTD, PPM, and safety stock	**Checked**
[x] API Key Protection	.env in .gitignore; placeholder in .env.example	**Checked**
[x] GitHub Repository	Source code committed and pushed to GitHub	**Checked**
[x] Demo Video	Google Drive video link included	**Checked**

26. Conclusion

The final application combines document retrieval, grounded question answering, source citations, and a deterministic policy calculator in one interface. I tested the required single-document questions, cross-document queries, Top-K behavior, persistence, trap-question handling, and policy thresholds. The implementation details, test results, setup steps, and demonstration video are documented in this report.

Developer Technical Report - HCLTech Assignment 2 Submission.